E-BOOK

CODERS.JAR TECH SUMMIT BS 2026

@coders.jar
CODERS.java Tech Summit BS 2026

Java Moderno: principais termos

Palestras e Painel

@coders.jar

01 Virtual Threads + LangChain4j: Escalabilidade e Performance em Sistemas Inteligentes

02 Do Código aos Cosmos Construindo Arquiteturas Escaláveis com Java

03 Evolução Java: Do Legado à era da IA

04 Além do provisionamento gerenciado | Java no mundo Serverless

05 Painel: O Papel da IA no Ecossistema Java e na Atuação dos Profissionais de Desenvolvimento"

Palestra 01

@coders.jar

Virtual Threads + LangChain4j: Escalabilidade e Performance em Sistemas Inteligentes

Virtual Threads

O que é?

São threads leves introduzidas no Java 21), gerenciadas pela JVM (Java Virtual Machine), não pelo sistema operacional, permitindo criar milhões de instâncias com baixo consumo de memória. Diferente das threads tradicionais (platform threads), você pode criar milhões delas sem esgotar a memória. Elas são a espinha dorsal do Projeto Loom, tornando a programação concorrente muito mais simples e escalável.

@coders.jar

Quais problemas resolve?

Elas revolucionam a concorrência ao resolver o alto custo das platform threads tradicionais, ideal para tarefas I/O bound.

- ***Alto consumo de recursos:*** *Threads tradicionais são "pesadas" e limitadas. Virtual Threads permitem alta concorrência com baixo consumo de memória e CPU.*
- ***Complexidade de código:*** *Elimina a necessidade de usar CompletableFuture ou programação reativa complexa para tarefas de I/O, permitindo um código mais imperativo e fácil de ler/debugar.*

Virtual Threads

Como aplicar no dia a dia?

Em qualquer aplicação que faça muitas operações de espera (I/O), como:

- chamadas a bancos de dados,
- APIs REST,
- leitura/escrita de arquivos ou filas.

Basta substituir a criação de threads tradicionais (***new Thread()***) ou o uso de um pool fixo por um ExecutorService que cria Virtual Threads: Executors.newVirtualThreadPerTaskExecutor().

- Exemplo: Um microsserviço que consulta 5 APIs diferentes para montar uma resposta pode disparar 5 Virtual Threads simultâneas, cada uma esperando sua resposta, sem travar o sistema.
- **API 1 - JSONPlaceholder:** API fake gratuita para testes (posts, usuários, comentários
  - **Caso de Uso**: Simular chamadas a múltiplos microsserviços.
- **API 2 - OpenWeatherMap**: Api de dados climáticos por cidade
  - **Caso de Uso:** Consultar previsão para múltiplas cidades
- **API 3** - **OpenAI API** (ChatGPT): LLM para processamento de linguagem natural
  - **Caso de Uso:** Resumir múltiplos documentos em paralelo
- **API 4** - **API Bancária Simulada:** Simulação de transferências/consultas
  - **Caso de Uso:**| Processar lote de transações

@coders.jar

LangChain4j

O que é?

É uma biblioteca de código aberto para Java que visa simplificar a integração de aplicações com Modelos de Linguagem de Grande Escala (LLMs). Ela segue os conceitos populares do framework LangChain (Python), mas para o ecossistema Java (LangChain**4J**).

@coders.jar

Quais problemas resolve?

- ***Complexidade de integração com IA:*** *Abstrai a complexidade de chamar APIs de LLM, gerenciar prompts, lidar com tokens e orquestrar fluxos de IA.*
- ***"Alucinação" de LLMs****: Ao combinar com seus dados (RAG), garante que as respostas sejam baseadas em fatos do seu domínio, aumentando a precisão e a confiabilidade.*
- ***Baixo acoplamento:*** *Permite trocar o provedor de LLM (ex: de OpenAI para Google Gemini) com poucas linhas de código.*

LangChain4j

Como aplicar no dia a dia?

- **Chatbots:**
  - Atendimento ao cliente
  - Assistentes virtuais.

- **Análise de Documentos:**
  - Resumo
  - Classificação e extração de informações de PDFs/textos.

Você pode carregar documentos (PDFs, CSVs, sites), "quebrá-los" em pedaços e armazená-los em um banco de dados vetorial.

- **Busca Semântica:**
  - Pesquisa de dados baseada no significado e não apenas em palavras-chave.

@coders.jar

COM LANGCHAIN4J VOCÊ CONSEGUE

- Criar fluxos para orquestrar em sequencia
- Usar memória para manter histórico da conversa
- Conectar a sua plicação a diferentes Provedores de IA
- IMPLEMENTAR RAG COM BANCO DE DADOS VEOTIRAS

RAG

O que é?

RAG (Retrieval Augmented Generation) **é uma técnica** de inteligência artificial que combina sistemas de recuperação de informações com modelos de linguagem generativos. Em vez de depender apenas do conhecimento interno do modelo (adquirido durante o treinamento), o RAG busca informações relevantes em uma base de dados externa antes de gerar uma resposta.

@coders.jar

Como funciona?

O processo básico do RAG tem três etapas:

1. **Recuperação (Retrieval):** Quando uma pergunta é feita, o sistema busca documentos ou trechos relevantes em uma base de conhecimento
2. **Augmentação (Augmentation)**: Os documentos recuperados são adicionados ao contexto da pergunta
3. **Geração (Generation):** O modelo de linguagem gera uma resposta baseada tanto na pergunta quanto nas informações recuperadas

RAG

O que é?

No desenvolvimento Java, RAG é especialmente útil para criar aplicações inteligentes que precisam acessar grandes volumes de informação.

Que problemas resolvem?

1. Chatbots e Assistentes Empresariais
Criar assistentes que respondem perguntas sobre documentação interna, políticas da empresa, ou bases de conhecimento técnico.

2. Análise de Documentação Técnica
Sistemas que ajudam desenvolvedores a encontrar informações em documentação extensa de APIs, frameworks ou código legado.

@coders.jar

3. Suporte ao Cliente Inteligente
Aplicações que consultam bases de FAQs, manuais de produtos ou histórico de tickets para fornecer respostas precisas.

Ferramentas Java para implementar RAG

- **LangChain4j:** A biblioteca mais popular para Java, oferece abstrações para trabalhar com LLMs e implementar RAG de forma simplificada.
- **Spring AI:** Framework que integra capacidades de IA, incluindo RAG, ao ecossistema Spring.
- **Vector Databases:** Integração com bancos como Pinecone, Weaviate, ChromaDB ou PostgreSQL com pgvector para armazenar embeddings.
- **Apache Lucene/Elasticsearch:** Para indexação e busca de documentos.

“Sistemas Inteligentes”

O que é?

No contexto de integração com IA, sistemas Java inteligentes são aplicações que incorporam capacidades de inteligência artificial para tomar decisões, processar linguagem natural, reconhecer padrões e ou automatizar tarefas complexas.
São aplicações completas que combinam lógica de negócio tradicional com capacidades de IA.

@coders.jar

Principais Características

- Processam dados não estruturados (texto, imagens, voz)
- Aprendem com dados históricos ou se adaptam a novos padrões
- Tomam decisões baseadas em modelos preditivos ou generativos
- Interagem de forma natural com usuários

“APIs Java Inteligentes”

O que é?

São interfaces que expõem funcionalidades de IA como serviços reutilizáveis: Ex.: ***REST APIs com IA:***

```
POST /api/analyze-document
POST /api/chat/completion
POST /api/classify-image
POST /api/translate
POST /api/summarize
```

@coders.jar

Principais Características

- *Encapsulam modelos de ML/IA complexos*
- *Oferecem endpoints simples para consumo*
- *Gerenciam contexto, estado e histórico quando necessário*
- *Lidam com rate limiting e escalabilidade*

Palestra 2

Do Código aos Cosmos

@coders.jar

Construindo Arquiteturas Escaláveis com Java

Arquitetura no Desenvolvimento de Um software

Antes, imagine a construção de uma casa?

@coders.jar

Análise do Terreno e Implantação (O Contexto)
- Topografia
- Orientação Solar e Ventilação
- Vistas e Paisagem

2. Funcionalidade e Programa de Necessidades (O Uso)
- Estilo de Vida
- Fluxo e Circulação
- Acessibilidade

3. Conforto Ambiental e Sustentabilidade (A Qualidade de Vida)
- Conforto Térmico e Acústico
- Iluminação Natural
- Sustentabilidade

4. Aspectos Técnicos e Estruturais (A Segurança)
- Qualidade Construtiva
- Legislação

5. Planejamento Financeiro (A Viabilidade)
- Estudo de Viabilidade (EVF)
- Planejamento de Etapas

6. Estética e Identidade (O Design)
- Estilo Arquitetônico
- Proporção e Materiais

No desenvolvimento de software é a mesma coisa, você irá construí-lo a partir de uma serie de requisitos, então você escolherá qual é a melhor arquitetura que se adéqua para aquela solução.

Arquitetura no Desenvolvimento de Um software

O que é?

É a estrutura fundamental de um sistema de software, ou seja, o conjunto de decisões de design mais importantes sobre como o software será organizado e construído.
Pense nela como a "planta" de um edifício/casa, mas para software. Assim como um arquiteto decide onde ficarão as paredes, portas e instalações elétricas de uma casa, o arquiteto de software decide como os diferentes componentes do sistema se conectam e trabalham juntos.

@coders.jar

Tipos de Arquiteturas em Java

Existem alguns tipos, vou deixar enumerados alguns deles aqui:

- MVC (Model View Controller)
- Monolito
  - Monolito Modular
  - Monolito Distribuído
  - Monolito de Camadas (Layered Monolith)
- Microservices
- **Hexagonal(Postas e Adaptadores)**
- **Serverless**
- Outros

Hexagonal é o que veremos nesta segunda palestra
Serverless também será usado na última palestra

MVC

O que é?

MVC (Model-View-Controller) é um padrão arquitetural que separa a aplicação em três componentes principais:
Model (Modelo): Representa os dados e a lógica de negócio. Gerencia o estado da aplicação, regras de negócio, validações e comunicação com o banco de dados. É independente da interface do usuário.
View (Visão): É a camada de apresentação, o que o usuário vê e interage. Exibe os dados do Model e envia as ações do usuário para o Controller. Pode ser HTML, JSON, XML, ou qualquer formato de saída.
Controller (Controlador): Atua como intermediário entre Model e View. Recebe as requisições do usuário, processa (geralmente delegando para o Model), e decide qual View deve ser renderizada como resposta.

@coders.jar

Quando Escolher MVC?

Mmuito usado hoje em dia no meio acadêmico

para pequenas aplicações.

Aplicações web tradicionais:

- Quando você está construindo aplicações web onde o servidor renderiza as páginas HTML (server-side rendering).
- Frameworks como Spring MVC.

Monolito

O que é?

É um estilo arquitetural onde toda a aplicação é construída e implantada como uma única unidade. Todo o código - interface do usuário, lógica de negócio, acesso a dados - roda no mesmo processo e é empacotado junto. Você compila, testa e implanta tudo de uma vez.

Quando implementar um monolito, quando o sistema não tiver muita escalabilidade, um número fixo de requisições, pouca integração com API's e "pouca" complexidade.

@coders.jar

Quando escolher Monolito?

Quando Escolher Monolito

- *Time pequeno (menos de 10-15 desenvolvedores)*
- *Produto em validação ou MVP*
- *Domínio de negócio simples e bem definido*
- *Requisitos de consistência forte entre diferentes partes*
- *Infraestrutura limitada ou time com pouca experiência em sistemas distribuídos*
- *Startup ou projeto novo onde velocidade de entrega é crítica*

Microservices

O que é?

Microsserviço é um estilo arquitetural onde a aplicação é construída como uma coleção de serviços pequenos e independentes, cada um executando em seu próprio processo e comunicando-se através de mecanismos leves (geralmente via protocolo **HTTP/REST** ou mensageria).

@coders.jar

Quando escolher?

- *Times grandes e distribuídos: digamos 5+ times*
- *Escalabilidade seletiva*
- *Ciclos de deploy diferentes*
- *Requisitos tecnológicos diversos: Uma parte precisa de processamento intensivo (melhor em Go), outra de ML (melhor em Python), outra é CRUD tradicional (melhor em Java).*
- *Isolamento de falhas*
- *Domínio de negócio complexo e bem delimitado*
- *Necessidade de deploys frequentes*

HEXAGONAL

O que é?

Arquitetura Hexagonal (também chamada de **Ports and Adapters**) é um padrão arquitetural que foca em isolar a lógica de negócio de tudo que é externo, como frameworks, bancos de dados, APIs, interface do usuário.

@coders.jar

Componentes principais:

- ***Domínio (centro do hexágono):*** *Contém as regras de negócio puras, entidades, casos de uso. Não depende de nada externo - sem anotações de frameworks, sem SQL, sem HTTP.*
- ***Portas (interfaces):*** *Definem os contratos de comunicação. Podem ser de entrada (quem chama a aplicação) ou de saída (o que a aplicação precisa chamar).*
- ***Adaptadores:*** *Implementações concretas das portas. Conectam o domínio ao mundo externo (REST controllers, repositórios de banco, clients HTTP, filas de mensagens).*

Quando escolher?

Lógica de negócio complexa, Necessidade de alta testabilidade, Múltiplas interfaces, Independência de tecnologia, Aplicações de longo prazo, Conformidade com Domain-Driven Design e Times grandes.

Palestra 3

Evolução Java: Do Legado à era da IA

@coders.jar

Java

O que é?

@coders.jar

Java é uma linguagem de programação orientada a objetos

- Plataforma (JVM)
- Linguagem

Características principais:

- **Multiplataforma:** Código Java compila para bytecode que roda na JVM (Java Virtual Machine), permitindo executar o mesmo programa em Windows, Linux, Mac, etc.
- **Orientada a objetos:** Tudo gira em torno de classes e objetos, promovendo reutilização e organização do código.
- **Fortemente tipada:** Você declara tipos de variáveis explicitamente, o compilador detecta muitos erros antes da execução.
- **Gerenciamento automático de memória:** Garbage collector cuida da limpeza de objetos não usados.
- **Rico ecossistema:** Bibliotecas, frameworks (Spring, Hibernate, Quarkus), ferramentas, e comunidade enorme.
- **Madura e estável:** 30 anos de evolução, extremamente confiável para produção.

Java

Como aplicar no dia a dia?

- **Sistemas empresariais (Enterprise)**;
- **Aplicações backend robustas:** APIs REST, microsserviços, processamento de transações. Java é excelente em performance sob alta carga sustentada;
- **Aplicações Android;**
- **Big Data e processamento distribuído**: Hadoop, Apache Spark, Apache Kafka, Elasticsearch - todos escritos em Java. Sistemas legados e migração;
- Times grandes e distribuídos;
- Aplicações de longa duração;
- Performance consistente;
- Setor financeiro e bancário;
- **Quando você precisa de ecossistema rico**: Spring (framework web), Hibernate (ORM), Maven/Gradle (build), JUnit (testes). Tudo muito maduro (Projeto, Time)

@coders.jar

Features LTS

O que é LTS?

LTS significa Long-Term Support (Suporte de Longo Prazo) - são versões especiais do Java que recebem suporte estendido por vários anos, incluindo patches de segurança, correções de bugs e atualizações de estabilidade.

@coders.jar

- **Versões LTS do Java:**
- **Java 8 (2014)** - LTS mais usado historicamente, suporte até 2030 em algumas distribuições
- **Java 11 (2018)** - Primeira LTS do novo modelo, muito adotada
- **Java 17 (2021)** - LTS atual mais popular, recomendada para novos projetos
- **Java 21 (2023)** - LTS mais recente, com features modernas
- **Java 25 (2025)** - Próxima LTS esperada

JEP

O que é LTS?

JEP significa JDK Enhancement Proposal (Proposta de Melhoria do JDK) - é o processo formal para propor, discutir e implementar mudanças significativas na plataforma Java.

@coders.jar

Como funciona:

- ***Proposta:*** *Alguém (geralmente da Oracle, mas pode ser da comunidade) identifica uma necessidade*
- *Rascunho: Escreve um JEP detalhando o problema, solução proposta, alternativas, riscos*
- *Revisão: Comunidade e especialistas revisam e comentam*
- *Aprovação: Se aprovado, vira alvo de implementação*
- *Implementação: Desenvolvido e integrado ao OpenJDK*
- *Release: Entregue em uma versão futura do Java*

Estrutura de um JEP:

- *Número único: JEP 395, JEP 409, etc.*
- *Título descritivo: "Records", "Pattern Matching for switch"*
- *Status: Draft, Candidate, Proposed to Target, Targeted, Integrated, Closed*
- *Descrição técnica: Motivação, especificação, alternativas, riscos*

Palestra 4

@coders.jar

Além do provisionamento gerenciado | Java no mundo Serverless

SERVERLEES

O que é?

Serverless é um modelo arquitetural para computação em nuvem onde você não gerencia servidores - você escreve apenas o código da sua aplicação e o provedor de nuvem cuida de toda a infraestrutura, escalabilidade, disponibilidade e manutenção.

@coders.jar

O nome é enganoso: *Servidores existem, mas você não se preocupa com eles. É "serverless" do seu ponto de vista como desenvolvedor.*

Duas categorias principais:

- ***FaaS (Function as a Service):*** *Você escreve funções individuais que são executadas em resposta a eventos. Exemplos: AWS Lambda, Google Cloud Functions, Azure Functions.*
- ***BaaS (Backend as a Service):*** *Serviços gerenciados que substituem componentes tradicionais. Exemplos: banco de dados (DynamoDB, Firestore), autenticação (AWS Cognito, Auth0), armazenamento (S3), filas (SQS).*

Como funciona (exemplo com AWS Lambda):

- *Você faz upload de uma função Java, configura um gatilho (HTTP via API Gateway, upload de arquivo no S3, mensagem em fila), e a função executa automaticamente quando o evento acontece. Você paga apenas pelo tempo de execução (milissegundos), não por servidor ocioso.*

OBSERVABILIDADE

O que é?

@coders.jar

É a capacidade de entender o que está acontecendo dentro de um sistema apenas observando suas saídas externas (métricas, logs, traces). Em outras palavras: conseguir responder *"por que meu sistema está com problema?"* sem precisar adicionar mais código ou fazer deploy.

Em serverless, uma requisição pode passar por 10+ serviços *(API Gateway → Lambda → SQS → Lambda → DynamoDB → Lambda → SNS)*.

Sem observabilidade adequada, você está voando às cegas.

Os 3 pilares da observabilidade:

- **Métricas:** Números agregados (requisições/segundo, latência p95, taxa de erro, custo)
- **Logs:** Eventos discretos (request recebido, query executada, erro aconteceu)
- **Traces:** Jornada completa de uma requisição através de múltiplos serviços

Painel

@coders.jar

O Papel da IA no Ecossistema Java e na Atuação dos Profissionais de Desenvolvimento

Bem-vestida, mas sem ter aonde ir

@coders.jar

- Futuro das profissões
- Profissões do Futuro
- IA no ecossistema Java
- Qual o papel dos profissionais de Desenvolvimento na era de IA?
- Os modelos de profissões do futuro na era de AI e programação;
- Futuro das Profissões de programação;
- Como nós prepararmos?
- Quais tecnologias e tendências precisamos saber ou até mesmo dominar neste cenário?
- Evolução, Riscos e Oportunidades para o Profissional da Programação e da tecnologia como um todo.

Aprenda os conceitos básicos, os fundamentos das linguagens e tecnologias, que a IA não irá tomar seu emprego.

Toda tecnologia começa com sentimento é isso que nos diferencia das máquinas".
Leonardo da Vinci

@coders.jar

Proximo Passos

Não esqueça, após o evento para reforçar o conhecimento faça pesquisa das palavras chaves, termos que você ais curtiu ou mais sentiu dificuldade.

- *Check-List dos principais aprendizados*
- *Check-List dos principais termos*
- *Check-List dos conteúdos com maiores dificuldades de absorver*

@coders.jar

REFERÊNCIAS

- *Documentações Oficiais*
- *https://javapro.io/2026/01/15/java-25-jep-512-jbang-notebooks-graallpy-and-raspberry-pi-for-interactive-learning/%20*
- *https://www.infoworld.com/article/3846172/jdk-25-the-new-features-in-java-25.html%20*
- *https://www.oracle.com/br/news/announcement/oracle-releases-java-25-2025-09-16/%20*
- *https://openjdk.org/projects/jdk/25/*

@coders.jar

Apoio

Parceiro